

A perfect storm

How a tiny change in a CMS once
broke up the Fashion Store?

Uri Savelchev

28.04.2026

zalando

01

The Catalog

This is the Fashion Store Catalog. It shows the list of products and allows to search them

02

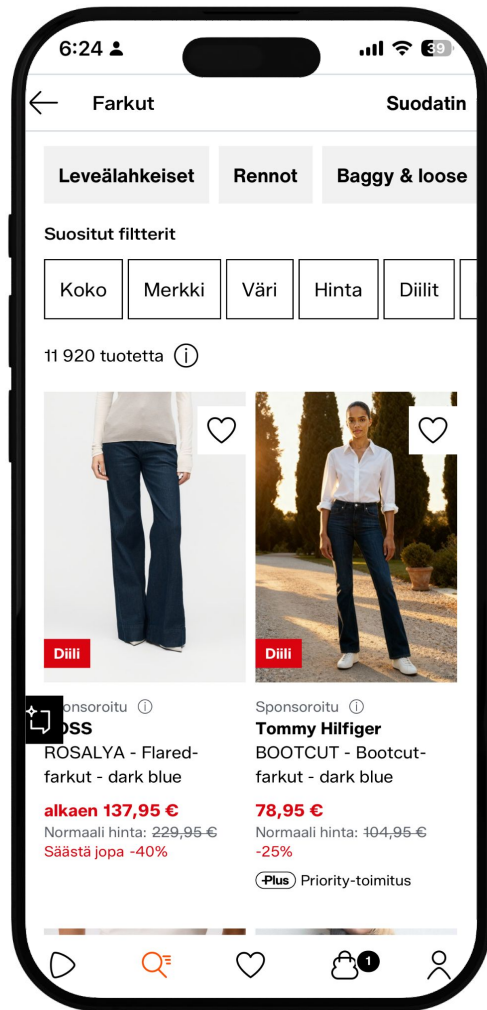
What happens if it doesn't work?

If the customers can't browse and select products, they won't buy anything.

03

Tier 1 service

The catalog and the search are vital for the whole Zalando Fashion Store.







Houston, we have a problem!

[redacted] 2024, 1:31 PM • Edited

There are 3 open alerts for catalog-api
The zmon alert details point to [this Catalog API grafana dashboard](#)

[redacted] 2024, 1:31 PM

Hi I m here from Search

[redacted] 2024, 2:05 PM

We ran some playbooks, the error rate is coming down, we're still working on it.

👍 1

[redacted] 2024, 2:07 PM

we dont know the root cause why elasticsearch pods continue having high CPU despite the scale up, we are running more emergency playbooks to stop the bleeding first...

[redacted] 2024, 2:19 PM

would it make sense to scale even further to give nodes more room to recover?

[redacted] 2024, 2:22 PM

its scaled to the max, we are changing the elasticsearch data nodes settings to let it scale up even more that will help bring down error rate meanwhile we are figuring out the root cause of high cpu on data nodes

👍 2



Firefighting

- The investigation identified that Elasticsearch was spending a lot of time to run facet aggregations.
- The team was in trying to stop the bleeding and applying mitigations based on symptoms, trying to mitigate the impact of those aggregations.
- The slow logs available in Elasticsearch could not be attributed to any specific client.
- The problem resolved itself around 19:30, but the analysis continued





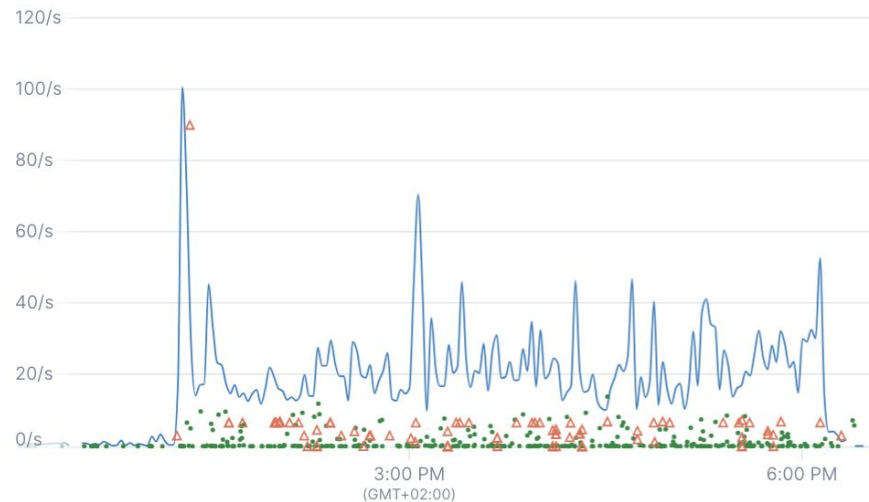
Finally, after 9 hours of analysis

Trace analysis shows higher activity of a CMS API on the same timeline as the incident. No evidence or correlation, but...

The offending traffic was low (20-100 RPS), with Catalog API traffic normally ranging between 8K-20K RPS.

Offending query patterns were analyzed and investigated. The prevailing pattern was huge aggregations on high cardinality fields (product IDs).

Next morning at 10:30 the incident re-occurred. The Catalog team has blocked requests from the CMS API and... it helped!



POSTMORTEM

```
graph TD; A[INCIDENT DATA ANALYSIS] --> B[ROOT CAUSE ISOLATION]; B --> C[PREVENTATIVE MEASURES]; B --> D[CORRECTIVE ACTIONS & LEARNINGS];
```

INCIDENT DATA ANALYSIS

ROOT CAUSE ISOLATION

PREVENTATIVE MEASURES

CORRECTIVE ACTIONS & LEARNINGS



"In complex systems, incidents are inevitable. Assessment and remediation are more involved and time-consuming, and it becomes increasingly costly to have the same recurring mistakes.

Performing Post-Mortems is an essential part of our efforts to drive operational excellence and collectively learn from past failures"

Zalando Incident Process

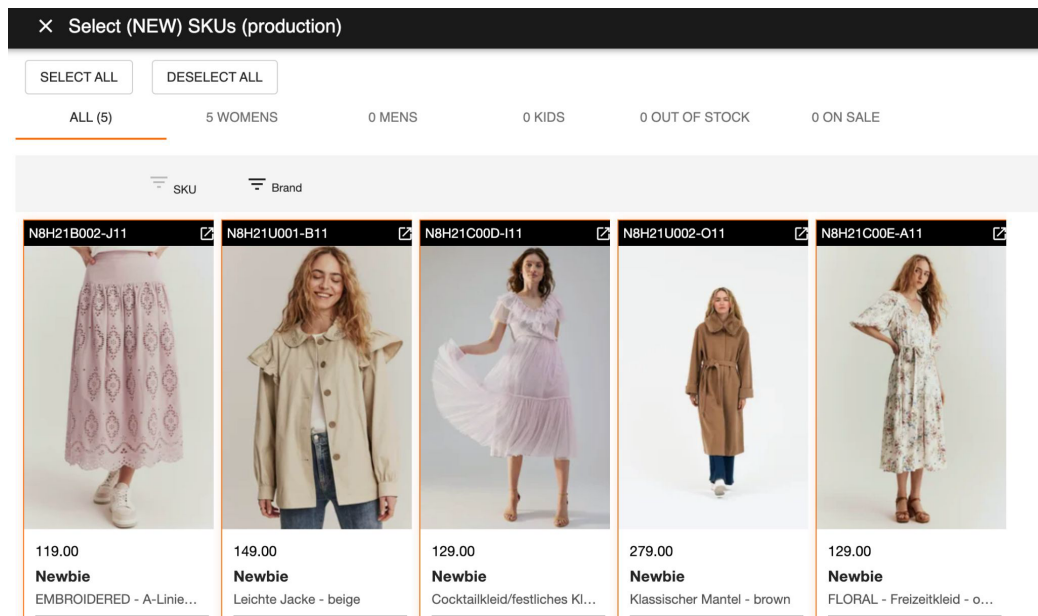
- Timeline
- Root cause analysis
- Detailed description
- Lessons learned
- Impact analysis
- Immediate actions
- Follow-up actions
- Risks



Why does the CMS API need to query Catalog API?

- Product discovery (e.g. for authoring collections and creating campaigns)
- Product validation (in-stock or out of stock)

but WHY it started sending the offending queries?





We hacked ourselves so you don't have to.

The traffic was caused by a security scan. The scan has identified an unprotected endpoint and started exploiting it, trying to figure out the impact.



A brief excursion into Zalando HTTPS handling

01

Skipper

An HTTP router and reverse proxy for service composition.

It can serve as a Kubernetes Ingress controller.

02

OAuth2 token validation

Skipper support OAuth2 token validation "on the edge" since 2018.

Before that all Zalando APIs validated tokens themselves - via querying the OAuth2 TokenInfo endpoint.

03

RouteGroup

A custom Kubernetes resource definition, an alternative to Ingress.

It allows route matching based on any HTTP request attributes, and provides a clean way for the request flow augmentation and traffic shaping.

Relies upon Skipper as an implementation.



Could you spot a mistake?

RouteGroup filter:

```
routes:
- pathSubtree: /
  filters:
    - oauthTokeninfoAnyKV("realm", "/employees")
    - oauthTokeninfoAnyKV("uid", "alejandro", "uid", "antti", "uid", "igor", "uid", "john", "victor")
    - forwardToken("x-tokeninfo", "uid", "scope", "realm")
```




Why did the mistake pass undetected?

01

Kubernetes API

Kubernetes API check has passed — the YAML follows the specification

02

RouteGroup Admission Controller

The admission controller checks have passed as well: the rules in the filter are syntactically correct

03

The problem is detected in runtime

The problem is detected in runtime when Skipper started to setup the filter

04

What did Skipper do?

Skipper ignored the whole route for consistency



How does the
service work if
there are no
routes defined
in the
RouteGroup?

It worked because an Ingress existed



01

The CMS has a long history

The CMS API did token validation itself before the function has been moved to Skipper

02

RouteGroup migration

At the same time the service migrated to use a RouteGroup

03

CI/CD

Zalando CI/CD does not remove K8s objects if they are removed from git. It should be done manually

04

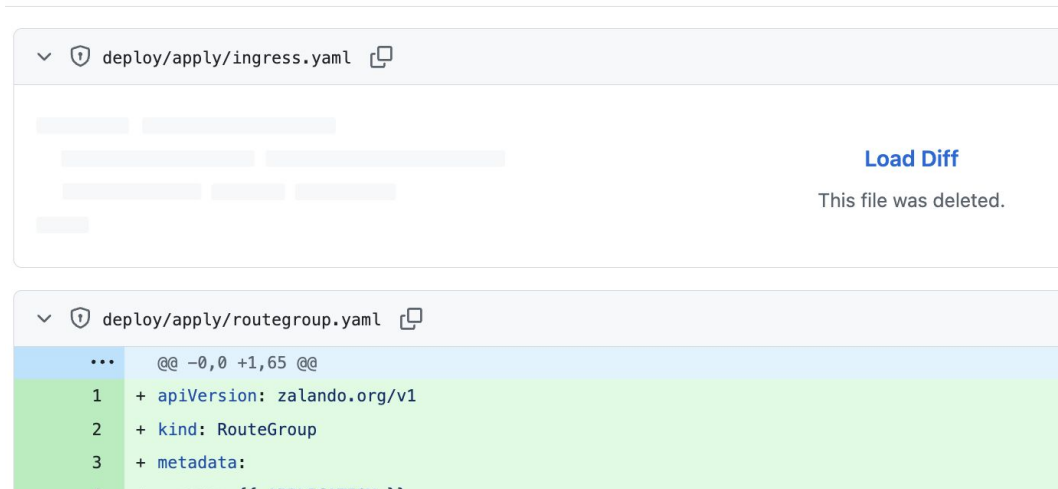
"Manual" means unreliable

The removal was missed and the old Ingress object lived in K8s *for several years*

05

Why it didn't affect anything?

In Skipper the most specific route always wins





How did the security scan create a load that affected the Catalog?

01

Amplification bug

The way how GraphQL query in the CMS API had been processed allowed to amplify the load, producing up to several thousands of requests to Catalog API from a single incoming request.

It never happened with legitimate CMS requests, but the security scanner applied random parameter values.

02

Inefficient query

The query was vastly inefficient: it requested faceting by an attribute with high cardinality. The faceting was not even used actually.



```
{
  "query": "query($param1: Int!, $param2: Int!, $param3: [String!]!) { searchArticles(size: $param1, limit: $param2, skus: $param3) { items { media { path, height, width }, name, sku, price, brand } } }",
  "variables": {
    "param1": 834673,
    "param2": 834673,
    "param3": [
      "sPedzS",
    ],
  }
}
```

01

Amplification bug

Even though the request contains the "skus" array, it also includes the "size" parameter that was used instead of the actual size of the array.

I observed this antipattern in GraphQL more than once.

JSON

```
{
  "offset": 6800,
  "limit": 200,
  "query": {
    "field": "sku",
    "terms": [ ... ]
  },
  "select": {
    "include": [ ... ],
    "exclude": []
  },
  "facets": {
    "category": {
      "type": "terms",
      "field": "sku",
      "include_empty": true,
      "offset": 0,
      "size": 956577
    }
  }
}
```

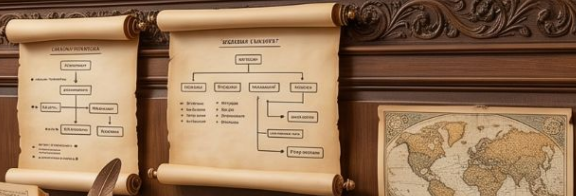
02

Inefficient query

The problem was never noticed before the incident due to very low request rate



LEARNINGS





A few learnings

"The incident surfaced the difficulty for the on-call team to pinpoint the root-cause of high load in Elasticsearch."

"Perfect storm of multiple factors and bugs in unconnected applications that in combination made the external traffic possible and amplified its effects"

"We should look for zebras as well as horses when we hear hoofbeats."

01

Security scan could be cause of a real incident. You need to have a playbook to shed the traffic.

02

KYC principle. Prepare to trace high load or slow queries back to their originators.

03

if you support an API with a complex query syntax, provide good documentations with examples and blueprints.

04

Do peer review of your APIs, whether they are small or big.

05

Don't ignore outdated Kubernetes resources - they can strike you back in an unexpected way.

Q

&

A